গভীর ডেক · D3

নিরাপত্তা

কে? কোন স্কুল? কী করতে পারে? — ৩২টি নিরাপত্তা-টেস্ট · ৩৬টি `PermissionDenied` · ৪টি ভূমিকা

মানচিত্র

নিরাপত্তা মানে তিনটি প্রশ্ন

প্রতিটি রিকোয়েস্টে তিনটি, আর প্রতিটির উত্তর দেওয়ার আলাদা জায়গা

১. কে?

লগইন হয়েছে? অ্যাকাউন্ট সক্রিয়? পাসওয়ার্ড সঠিক?

উত্তরদাতা: Django-র Auth মিডলওয়্যার + নিজের User মডেল

ব্যর্থ হলে: লগইন পাতায় রিডাইরেক্ট

২. কোন স্কুল?

এই ইউজার এই workspace-এর সদস্য? স্কুলটি সক্রিয়?

উত্তরদাতা: `workspace_view` ডেকোরেটর

ব্যর্থ হলে: 404 — স্কুলটির অস্তিত্বই স্বীকার নয়

৩. কী করতে পারে?

ভূমিকা অনুমোদিত? ক্লাস/ছাত্র তার পরিধিতে?

উত্তরদাতা: role set + `scoping.py`

ব্যর্থ হলে: 403 বা খালি তালিকা

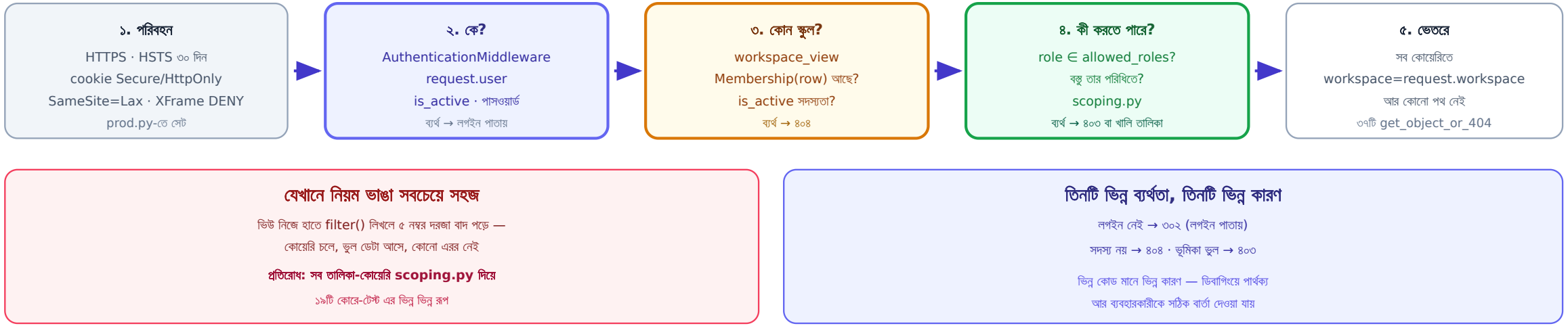
তিনটি প্রশ্নের উত্তর **তিন জায়গায়** — এটাই নকশা। এক জায়গায় গুঁজে দিলে একটি ফাঁক সব কিছু ফাঁকা করে দিত।

শিক্ষকের নোট: এই তিনটি প্রশ্ন বোর্ডে লিখুন এবং সারাক্ষণ ফিরে আসুন। শিক্ষার্থীরা প্রায়ই দ্বিতীয়টি ভুলে যায় — আর মাল্টি-টেন্যান্ট অ্যাপের সবচেয়ে বিপজ্জনক বাগ ঠিক সেখানেই হয়।

স্তর

একটি অননুমোদিত রিকোয়েস্ট চারটি দরজা পেরোয়

প্রতিটি দরজা আলাদা প্রশ্ন, আলাদা উত্তর, আলাদা ব্যর্থতা



শিক্ষকের নোট: পঞ্চম দরজাটি ব্যাখ্যা করুন — প্রতিটি কোয়েরিতে workspace শর্ত। এটি লেখা থাকে ভিউতে, তাই এখানেই ভুল হয়। বাঁচার উপায় একটাই: হাতে ফিল্টার না লেখা।

স্তর ২-৩

`workspace_view` — চারটি ধাপ, ২৮ লাইন

৫২টি ভিউ এই একটি ডেকোরেটরের উপর দাঁড়িয়ে

```
if not request.user.is_authenticated:
    return redirect(f"/accounts/login/?next={request.path}")

workspace = Workspace.objects.filter(
    slug=slug, is_active=True).first()
if workspace is None:
    raise Http404("Workspace not found.")

membership = get_membership(request.user, workspace)
if membership is None:
    raise Http404("Workspace not found.")   # ইচ্ছাকৃত

if allowed_roles and membership.role not in allowed_roles:
    raise PermissionDenied("Your role ... does not allow this action.")

request.workspace = workspace
request.membership = membership
request.session["active_workspace_id"] = workspace.pk
```

ধাপ	ফল
১. লগইন?	না হলে ৩০২ + `?next=` — কাজ হারায় না
২. স্কুল আছে?	না হলে ৪০৪
৩. সদস্য?	না হলে **৪০৪**, ৪০৩ নয়
৪. ভূমিকা?	ভুল হলে ৪০৩ — বার্তা সহ
সফল হলে	`request.workspace` + সেশন আপডেট

প্ল্যাটফর্ম-অ্যাডমিনও ছাড় পায় না: ডেকোরেটরের মন্তব্যে লেখা — "platform admin may look at platform-level pages, never at private records of a workspace they do not belong to."

শিক্ষকের নোট: ৩য় ধাপে থামুন। জিজ্ঞেস করুন: "সদস্য না হলে ৪০৩ দিলে কী ক্ষতি?" উত্তর: ৪০৩ মানে "এই স্কুল আছে, কিন্তু তোমার অধিকার নেই" — সেটাই তথ্য ফাঁস।

প্রমাণ

কেন ৪০৪ বনাম ৪০৩ — এবং টেস্ট যা তা প্রমাণ করে

প্রতিটি কোডের পেছনে একটি টেস্টের নাম আছে

টেস্ট (আসল নাম)	প্রত্যাশা	কেন এটাই সঠিক
`test_non_member_gets_404_not_403`	৪০৪	সদস্যপদ নেই মানে স্কুলটির অস্তিত্ব জানানোর অধিকারও নেই
`test_anonymous_visitor_is_sent_to_login`	৩০২	লগইন নেই — লগইনই সমাধান, তাই সেখানেই পাঠানো
`test_student_cannot_open_teaching_pages`	৪০৩	সদস্য, কিন্তু ভূমিকা ভুল — "কেন নয়" বলা নিরাপদ
`test_coordinator_cannot_open_owner_only_pages`	৪০৩	সমন্বয়ক দেখে, কিন্তু সেটিং বদলায় না
`test_teacher_cannot_open_administration_pages_by_typing_the_url`	৪০৩	মেনুতে লিংক নেই — তবু URL টাইপ করে ঢোকা যায় না
`test_inactive_membership_sees_nothing`	৪০৪ / খালি	সদস্যতা বন্ধ করা মানে অপসারণ, কিন্তু ডেটা রাখা

নিয়ম: প্রতিটি অনুমতি-সিদ্ধান্তের একটি টেস্ট থাকা চাই — নাহলে ভবিষ্যতের কেউ ডেকোরেটর বদলে দিলে কেউ টের পাবে না।

শিক্ষকের নোট: "মেনুতে লিংক নেই" আর "ঢোকা যায় না" — এই দুইয়ের পার্থক্য শিক্ষার্থীদের প্রায়ই বুঝতে হয় না। UI লুকানো নিরাপত্তা নয়; সার্ভার-চেকই নিরাপত্তা।

পরিধি

`scoping.py` — এক জায়গায় "কে কী দেখবে"

তালিকা-কোয়েরি আর বস্তু-যাচাই একই ফাংশন ডাকে, তাই কখনো মতভেদ হয় না

```
def visible_students(user, workspace, membership=None):
    qs = StudentProfile.objects.filter(workspace=workspace)
    if membership is None:
        return qs.none()

    if role == ORG_ADMIN:
        return qs                          # পুরো স্কুল

    if role in (COORDINATOR, TEACHER):
        return qs.filter(
            enrollments__is_active=True,
            enrollments__class_group__in=
                visible_classes(user, workspace)).distinct()

    if role == STUDENT:
        return qs.filter(pk=student.pk)  # কেবল নিজের

    return qs.none()
```

ফাংশন	কী ফেরত দেয়
`visible_classes`	ভূমিকা অনুযায়ী ক্লাসের queryset
`visible_students`	ওই ক্লাসগুলোর সক্রিয় ছাত্র
`visible_exams`	ছাত্রের জন্য কেবল `is_published` + নিজের নম্বর
`visible_homework`	ছাত্রের জন্য কেবল প্রকাশিত
`teacher_can_access_class()`	একই ফাংশনের `.filter(pk=…).exists()`

কারণ: **তালিকা ও বস্তু যাচাই আলাদা লজিক হলে** একদিন একটি পথ খোলা থেকে যায় — তালিকায় দেখা যায় না, কিন্তু URL জানলে ঢোকা যায়।

শিক্ষকের নোট: এই ফাইলটিই ডেকের হৃদয়। প্রশ্ন করুন: "নতুন ভূমিকা যোগ করলে কত জায়গায় বদলাতে হবে?" উত্তর: এক জায়গায় — `scoping.py`. এটাই ভূমিকা-বাগ একবার ঠিক করার নকশা।

ম্যাট্রিক্স

চারটি ভূমিকা, আটটি কাজ

প্রতিটি ঘর কোডে লেখা — ডকুমেন্টে নয়, তাই পুরনো হওয়ার সুযোগ নেই

কাজ	ORG_ADMIN	COORDINATOR	TEACHER	STUDENT
হাজিরা নেওয়া	✓	✓	✓	×
নম্বর এন্ট্রি	✓	✓	✓	×
ফলাফল প্রকাশ	✓	✓	✓ (নিজের ক্লাস)	×
শ্রেণি/বিষয় তৈরি	✓	×	×	×
শিক্ষক-অ্যাসাইনমেন্ট	✓	×	×	×
অডিট লগ দেখা	✓	×	×	×
ব্র্যান্ডিং / সেটিং	✓	×	×	×
নিজের রেজাল্ট দেখা	✓	✓	✓	✓ (কেবল নিজের)

কোডে: `OWNERS` (১টি) · `MANAGERS` (২টি) · `TEACHING` (৩২টি ভিউ) · `ALL_ROLES` (৮টি ভিউ)

সূক্ষ্মতা: TEACHER "ফল প্রকাশ" পারে, কিন্তু কেবল `teacher_can_access_class()` পাশ করলে — ভূমিকা ঠিক, বস্তু ভুল, তখন ৪০৩।

শিক্ষকের নোট: ম্যাট্রিক্সটি বোর্ডে আঁকুন এবং শিক্ষার্থীদের জিজ্ঞেস করুন: "সমন্বয়ক নম্বর দিতে পারে, কিন্তু শ্রেণি বানাতে পারে না — কেন?" উত্তর: নম্বর তার দৈনন্দিন কাজ, শ্রেণি-কাঠামো প্রতিষ্ঠানের সিদ্ধান্ত।

পরিধি-২

সমন্বয়কের পরিধি — অনুমতি ডেটা হিসেবে

`CoordinatorScope` · "এই দুই বিভাগ আর এই এক ক্লাস"

```
class CoordinatorScope(models.Model):
    workspace = FK(Workspace)
    user = FK(User)
    department = FK(Department, null=True)
    class_group = FK(ClassGroup, null=True)
    is_active = Bool(default=True)

    class Meta:
        constraints = [CheckConstraint(
            condition=~(Q(department__isnull=True) &
                        Q(class_group__isnull=True)),
            name="coordinator_scope_needs_a_target")]
        indexes = [Index(fields=["workspace", "user", "is_active"])]
```

সিদ্ধান্ত	কারণ
রোল-স্ট্রিং নয়, সারি	"সমন্বয়ক-২" বানালে প্রতিটি নতুন স্কুলে নতুন কোড লাগত
`department` বা `class_group`	কেউ বিভাগভিত্তিক, কেউ এক ক্লাসের — দুটোই সম্ভব
`CheckConstraint`	দুটোই null মানে "কিছুই দেখবে না" — অর্থহীন সারি, তাই ডেটাবেজেই বন্ধ
`is_active` + ইনডেক্স	পরিধি বাতিল মানে মুছে ফেলা নয়; কোয়েরি indexed

প্রমাণ: `test_coordinator_sees_only_their_scope` — দুই সমন্বয়ক, আলাদা পরিধি, একে অন্যের ক্লাস দেখে না।

সৎ সীমা: পরিধি একই স্কুলের ভেতরে সীমিত করে, কিন্তু ডেটাবেজ-স্তরে (RLS) নয়। কেউ ভিউতে `scoping.py` বাদ দিলে ডেটাবেজ তাকে আটকাবে না।

শিক্ষকের নোট: "অনুমতি ডেটা হিসেবে রাখা" — এই বাক্যটি মনে রাখতে বলুন। এর অর্থ: নতুন পরিধি যোগ করতে মাইগ্রেশন লাগে না, কেবল একটি সারি।

স্তর ৪

ভূমিকা ঠিক, বস্তু ভুল — সবচেয়ে সূক্ষ্ম ফাঁক

শিক্ষক একজন, কিন্তু ক্লাস ৪০টি; ডেকোরেটর কেবল প্রথমটি জানে

```
@workspace_view(*TEACHING)
def exam_detail(request, slug, pk):
    exam = get_object_or_404(Exam, pk=pk,
                             workspace=request.workspace)
    if not teacher_can_access_class(request.user,
                                    exam.class_group):
        raise PermissionDenied(
            "You are not assigned to this class.")
```

একই তিন লাইন ভিউতে যেখানে ক্লাস/ছাত্র/পরীক্ষা জড়িত — নাহলে পরিধি বাইরে যাওয়া যায়।

স্তর	যা জানে
১. সদস্যতা	ইউজার এই স্কুলের সদস্য
২. ভূমিকা	তার ভূমিকা এই কাজটি করতে পারে
৩. বস্তু	এই নির্দিষ্ট ক্লাস/ছাত্র তার পরিধিতে

ফাঁদ: ২ নম্বর পাশ করলেই সব ক্লাস খুলে যায় — যদি ৩ নম্বর না থাকে। শিক্ষক ৭ম শ্রেণির, কিন্তু URL-এ ৯ম শ্রেণির `pk` বসালে দেখতে পাবেন।

প্রমাণ: `test_teacher_sees_only_assigned_classes` · `test_teacher_only_sees_students_of_their_own_classes` · `test_teacher_with_no_assignment_sees_no_students`

শিক্ষকের নোট: এটি IDOR (insecure direct object reference) — নামটি বলুন, কারণ ইন্টারভিউতে জিজ্ঞেস করা হয়। উদাহরণ: নিজের রেজাল্ট পাতার URL-এ অন্যের `pk` বসানো।

৩২ টেস্ট

প্রতিটি স্তরের প্রমাণ — নাম ধরে

১০৩টি টেস্টের মধ্যে ৩২টি সরাসরি নিরাপত্তার (৩১%)

স্তর ১–২ (১০টি)	প্রমাণ করে
`anonymous_visitor_is_sent_to_login`	লগইন ছাড়া কিছু নয়
`dashboard_requires_login`	টাইপ করা URL-ও নয়
`deactivated_user_cannot_log_in`	অ্যাকাউন্ট বন্ধ = ঢোকা বন্ধ
`wrong_password_fails`	ভুল পাসওয়ার্ড পরিষ্কার ব্যর্থ
`non_member_gets_404_not_403`	স্কুলের অস্তিত্ব ফাঁস নয়
`inactive_membership_sees_nothing`	সদস্যতা বাতিল = দেখা বন্ধ
`verified_domain_shows_that_institution…`	ডোমেইন → সঠিক স্কুল
`unverified_domain_is_ignored`	যাচাই না হলে উপেক্ষা
`domain_cannot_be_stolen_by_second…`	ডোমেইন দখল অসম্ভব
`only_the_platform_admin_can_open_console`	প্ল্যাটফর্ম আলাদা

স্তর ৩–৪ (২২টি)	প্রমাণ করে
`teacher_cannot_open_administration…`	URL টাইপেও নয়
`student_cannot_open_teaching_pages`	ভূমিকা = দেয়াল
`coordinator_cannot_open_owner_only_pages`	দেখা ≠ বদলানো
`each_role_is_offered_only_its_own_links`	মেনুও মিথ্যা বলে না
`teacher_sees_only_assigned_classes`	পরিধি কার্যকর
`coordinator_sees_only_their_scope`	ডেটা-ভিত্তিক পরিধি
`student_sees_only_own_record`	নিজের বাইরে নয়
`student_cannot_see_another_students_marksheet`	IDOR বন্ধ
`results_are_invisible_until_published`	প্রকাশ-নিয়ন্ত্রণ
`publishing_is_audited` · `correction_requires_a_reason…`	প্রতিটি বদল লিপিবদ্ধ

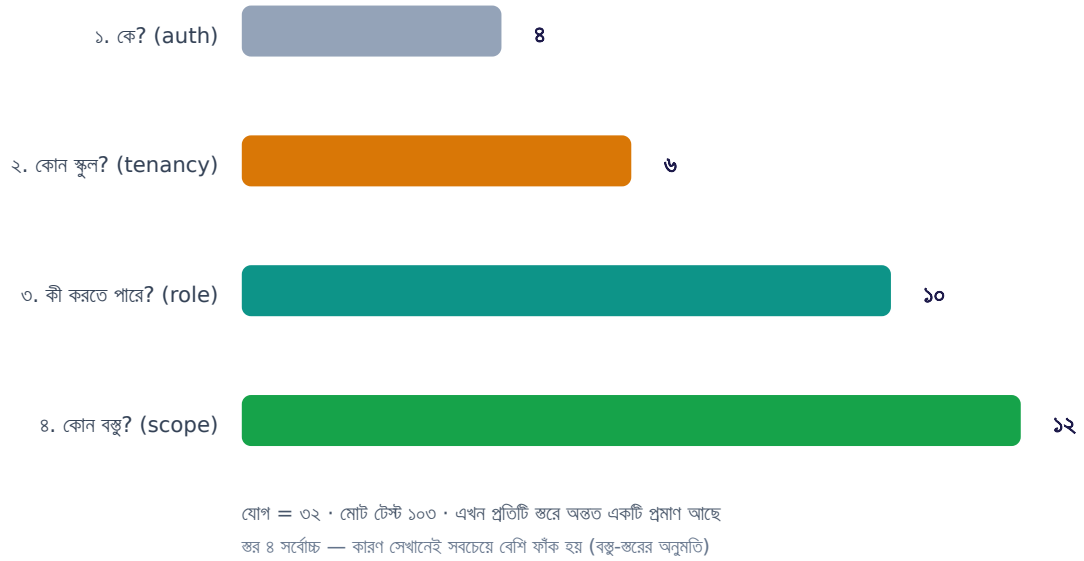
বাকি ৭১টি টেস্ট কার্যকারিতা যাচাই করে (হাজিরা সেভ হয়, নম্বর যোগ হয়) — নিরাপত্তার প্রমাণ নয়। দুই ধরনই দরকার।

শিক্ষকের নোট: এই দুই কলাম একসাথে পড়ুন — এটাই "নিরাপত্তা ডকুমেন্টে লেখা থাকে না, টেস্টে থাকে" এর প্রমাণ। শিক্ষার্থীদের নিজের প্রজেক্টে এমন একটি তালিকা বানাতে দিন।

বণ্টন

৩২টি টেস্ট কোন স্তরে কত

কম সংখ্যা মানে দুর্বলতা নয় — কিন্তু দেখতে হবে কোথায়



যোগ = ৩২ · মোট টেস্ট ১০৩ · এখন প্রতিটি স্তরে অন্তত একটি প্রমাণ আছে

স্তর ৪ সর্বোচ্চ — কারণ সেখানেই সবচেয়ে বেশি ফাঁক হয় (বস্তু-স্তরের অনুমতি)

যা এখনো প্রমাণহীন

রেট-লিমিট নেই → টেস্টও নেই

CSRF ব্যর্থতার পাতা

আপলোডের আকার/ধরন যাচাই

prod-এর নিরাপত্তা-হেডার

সেশন-মেয়াদ শেষ হওয়া

স্তর ৪ কেন সবচেয়ে বড়

একটি ভূমিকা-চেক সহজ, কিন্তু

৪০টি ক্লাসের প্রতিটির অনুমতি

আলাদা করে ভাবতে হয়

তাই সেখানে ১২টি টেস্ট

শিক্ষকের নোট: ডান পাশের লাল বাক্সটি আগে পড়ুন — সততাই এখানে মূল শিক্ষা। "কত টেস্ট" নয়, "কোন প্রশ্নের উত্তর নেই" — সেটাই আসল মাপ।

পাসওয়ার্ড

চারটি ভ্যালিডেটর — আর একটি ইচ্ছাকৃত সীমা

`AUTH_PASSWORD_VALIDATORS` · ন্যূনতম ১০ অক্ষর

ভ্যালিডেটর	কী আটকায়
UserAttributeSimilarity	নাম/ইমেইলের কাছাকাছি পাসওয়ার্ড
MinimumLength **min_length=10**	ছোট পাসওয়ার্ড
CommonPassword	"password123" জাতীয় তালিকা
NumericPassword	কেবল সংখ্যা — "01712345678" নয়

ডেমো পাসওয়ার্ড `DemoPass!2026` — ১০ অক্ষরের বেশি, তালিকায় নেই, সংখ্যা-একক নয়।

সৎ সীমা: "কতবার ভুল হয়েছে" গণনা কোথাও নেই — ১০০ বার ভুল পাসওয়ার্ড দিলেও কিছু হয় না। brute-force ঠেকানোর কোনো স্তর নেই।

ভালো দিক: লগইনে ভুল হলে বার্তা একটাই — "E-mail or password is incorrect." ফলে কোন ইমেইল আছে, তা বোঝা যায় না (user enumeration বন্ধ)।

কিন্তু: নিষ্ক্রিয় অ্যাকাউন্টে আলাদা বার্তা — "This account is deactivated." এতে বোঝা যায় অ্যাকাউন্টটি আছে। ব্যবহারকারীর সুবিধার জন্য নেওয়া ছাড়, তথ্য ফাঁসের সামান্য দাম।

শিক্ষকের নোট: শেষ পয়েন্টটি সূক্ষ্ম — নিরাপত্তা আর সহায়তার মধ্যে এই ধরনের ছাড় প্রতিটি অ্যাপে থাকে। শিক্ষার্থীদের জিজ্ঞেস করুন: "কোনটি বেছে নিতেন?" — সঠিক উত্তর নেই, কিন্তু সিদ্ধান্তটা জানা দরকার।

লগইন

লগইনে তিনটি ভালো সিদ্ধান্ত, একটি বড় ফাঁক

`IMSLoginView` · ৯ লাইন কোড, বড় প্রভাব

```
def form_valid(self, form):
    # আগের সেশনের workspace মুছে ফেলো:
    # এই ব্রাউজারে অন্য কেউ লগইন করেছিল
    self.request.session.pop(SESSION_KEY, None)
    response = super().form_valid(form)
    messages.success(self.request, "Welcome back!")
    return response
```

সিদ্ধান্ত	কেন
সেশন-কী মুছে ফেলা	শেয়ার্ড কম্পিউটার: আগের ব্যবহারকারীর স্কুলে নতুন ব্যবহারকারী ঢুকে যেত না
`redirect_authenticated_user`	লগইন থাকা অবস্থায় লগইন পাতায় গেলে ড্যাশবোর্ডে ফেরত
Django-র `login()`	সেশন-কী ঘোরায় (session fixation ঠেকায়) — নিজে লিখতে হয় না

বড় ফাঁক: সফল বা ব্যর্থ লগইন — কোনোটিই `AuditLog`-এ যায় না। "কে কখন ঢুকল" প্রশ্নের উত্তর ডেটাবেজে নেই, কেবল সার্ভার লগে (যা ঘুরিয়ে ফেলা যায়)।

শিক্ষকের নোট: সেশন-কী মুছে ফেলার লাইনটি সবচেয়ে বড় শিক্ষা — ছোট কোড, বড় ঝুঁকি বন্ধ। জিজ্ঞেস করুন: "এটি না থাকলে কী হতো?" উত্তর: ল্যাবে একজন লগআউট না করে গেলে পরেরজন তার স্কুলে ঢুকে যেত।

সেশন

সেশন ও কুকি — ছয়টি পতাকা, একটি কারণ

প্রতিটি পতাকা একটি নির্দিষ্ট আক্রমণ ঠেকায়

সেটিং (prod)	মান	যা ঠেকায়
`SESSION_COOKIE_SECURE`	True	HTTP-তে কুকি যাওয়া — নেটওয়ার্কে আটকানো
`SESSION_COOKIE_HTTPONLY`	True	JavaScript দিয়ে কুকি পড়া (XSS-এর ক্ষতি সীমিত)
`SESSION_COOKIE_SAMESITE` · `CSRF_COOKIE_SAMESITE`	Lax	অন্য সাইট থেকে আসা POST-এ কুকি পাঠানো
`SECURE_SSL_REDIRECT`	True	সব HTTP → HTTPS
`SECURE_HSTS_SECONDS`	৩০ দিন	ব্রাউজারকে "এই সাইট কেবল HTTPS" মনে রাখানো
`X_FRAME_OPTIONS`	DENY	অন্য সাইটে iframe-এ বসিয়ে ক্লিকজ্যাকিং

মেয়াদ ৮ ঘণ্টা + `SESSION_SAVE_EVERY_REQUEST` — কাজ করলে মেয়াদ বাড়ে, ফেলে রাখলে শেষ হয়।

যা নেই: CSP হেডার নেই (`django-csp` ইনস্টল নয়) — তাই XSS ঘটলে ব্রাউজার-স্তরে বাধা নেই; ভরসা কেবল template autoescape।

শিক্ষকের নোট: শিক্ষার্থীরা HSTS-এর মানে জানে না। ব্যাখ্যা: একবার HSTS পাঠালে ব্রাউজার ৩০ দিন HTTP-তেই সংযোগ করবে না — অর্থাৎ ভুল সেট করলে ফিরিয়ে আনা কঠিন।

CSRF

CSRF — টোকেন, উৎস, আর একটি ত্রুটি-পাতা

ডিফল্ট `CsrfViewMiddleware` + নিজের বার্তা

```
CSRF_FAILURE_VIEW = "apps.core.views.csrf_failure"
CSRF_TRUSTED_ORIGINS = env_list(
    "CSRF_TRUSTED_ORIGINS",
    "http://127.0.0.1:8000,http://localhost:8000,"
    "https://*.e2b.app")

def csrf_failure(request, reason=""):
    return render(request, "403_csrf.html",
        {"reason": reason,
         "origin": request.META.get("HTTP_ORIGIN","")},
        status=403)
```

প্রশ্ন	উত্তর
আক্রমণ কেমন?	অন্য সাইটে লুকানো ফর্ম → আপনার ব্রাউজারের কুকি দিয়ে POST
প্রতিরোধ কী?	ফর্মে টোকেন, যা অন্য সাইট জানে না
ব্যর্থ হলে?	ডিফল্ট ৪০৩-এর বদলে বাংলা/ইংরেজি ব্যাখ্যা-পাতা, উৎস দেখা যায়
কেন `TRUSTED_ORIGINS`?	HTTPS প্রক্সি/স্যান্ডবক্স ডোমেইন থেকে আসা ফর্ম বৈধ

সৎ ফাঁক: `csrf_failure` পাতার কোনো টেস্ট নেই — কেউ ভাঙলে টের পাওয়া যাবে ব্যবহারকারীর অভিযোগে।

মাপা হিসাব: ৩৫টি টেমপ্লেটে ফর্ম আছে · ২৭টিতে `{% csrf_token %}` · বাকি ৮টি `method="get"` ফিল্টার — সেখানে টোকেন লাগেই না। অর্থাৎ প্রতিটি POST ফর্মে টোকেন আছে।

শিক্ষকের নোট: বোর্ডে তিন লিখুন: কুকি (কে), টোকেন (এই ফর্ম), Origin (কোন সাইট)। তিনটি না মিললে ৪০৩ — এটাই CSRF-এর পুরো গল্প।

হোস্ট

কাস্টম ডোমেইন দখল করা কেন কঠিন

তিনটি বাধা: যাচাই, অনন্যতা, আর "নিজে সেট করা যায় না"

বাধা	কোথায়
যাচাই করা ছাড়া কাজ করে না	`filter(domain_verified=True)` — মিডলওয়্যারে
এক ডোমেইন একবারই	`UniqueConstraint(name="unique_custom_domain")`
মালিক নিজে বসাতে পারেন না	প্ল্যাটফর্ম-অ্যাডমিনই সেট করেন — কারণ ডোমেইন যাচাই প্রমাণ দরকার

প্রমাণ: `test_owner_cannot_set_a_custom_domain_themselves` · `test_domain_cannot_be_stolen_by_a_second_institution`

যে ঝুঁকিটা লেখা হয়েছে: মিডলওয়্যার রানটাইমে `settings.ALLOWED_HOSTS`-এ ডোমেইন যোগ করে। এটি কাজ করে, কিন্তু অ্যাপের গ্লোবাল অবস্থা বদলায় — একই প্রসেসে অন্য রিকোয়েস্টও এই তালিকা দেখে।

ভালো দিক: `patch_vary_headers(response, ["Host"])` — হোস্টভেদে আলাদা ক্যাশ, নাহলে এক স্কুলের লগইন পাতা আরেক স্কুলে দেখাত।

সংশোধনের পথ: তালিকা বদলে রিকোয়েস্টে পাস করা, বা `sync_domains` কমান্ড দিয়ে আগেই তালিকা তৈরি রাখা।

শিক্ষকের নোট: "গ্লোবাল সেটিং রানটাইমে বদলানো" — এটি প্রায়ই বাগের উৎস। প্রশ্ন করুন: "দুই স্কুলের অনুরোধ একসাথে এলে কী হয়?" উত্তর: সাধারণত কাজ করে, কিন্তু নিশ্চয়তা নেই।

ফাইল

আপলোড করা ফাইল — সবচেয়ে দুর্বল জায়গা

লোগো `ImageField`, ম্যাটেরিয়াল `FileField` — দুটির সুরক্ষা সমান নয়

জিনিস	যা যাচাই হয়
`WorkspaceBranding.logo`	`ImageField` → Pillow ছবি কি না যাচাই করে (অ-ছবি আপলোড ব্যর্থ)
`Material.file`	**কিছুই না** — শুধু ফাইল-নাম আর পথ (`materials/%Y/%m/`)
আকার-সীমা	কোনো `FILE_UPLOAD_MAX_MEMORY_SIZE` কাস্টম নেই
ধরন-সীমা	PDF/DOCX নয় এমন ফাইলও গ্রহণযোগ্য

ঝুঁকি: কেউ `.html` বা `.svg` আপলোড করে লিংক শেয়ার করলে ব্রাউজারে স্ক্রিপ্ট চলতে পারে — যদি মিডিয়া ফাইল পাবলিক পথে সার্ভ হয়।

এখন যা হয়: `/media/` কেবল `DEBUG=True`-তে সার্ভ হয় (`config/urls.py`)। প্রোডাকশনে মিডিয়া ওয়েব-সার্ভারকে দিতে হয় — এবং সেই নিয়ম কোথাও লেখা নেই।

সংশোধন: অনুমোদিত এক্সটেনশন-তালিকা + আকার-সীমা + `Content-Disposition: attachment`। এখনো বাকি।

শিক্ষকের নোট: "ফাইল আপলোড মানে ব্যবহারকারীর দেওয়া কোড আপনার সার্ভারে" — এই বাক্যটি শেখান। Student-এর দেওয়া হোমওয়ার্ক ফাইলও একই ঝুঁকি বহন করে।

অডিট

২৩ ধরনের কাজ লিপিবদ্ধ — আর একটি নীরব ফাঁদ

`audit.log()` কখনো রিকোয়েস্ট ভাঙে না — কিন্তু তাই কিছু চুপচাপ হারায়ও যায়

```
def log(request, action, obj=None, ...):
    try:
        return AuditLog.objects.create(
            workspace=..., actor=request.user,
            action=action, before=..., after=...,
            ip_address=request.META["REMOTE_ADDR"])
    except Exception:      # noqa: BLE001
        return None        # অডিট কখনো ভাঙবে না
```

দাম: লগ লেখা ব্যর্থ হলে কেউ জানে না — ডেটাবেজ ভরে গেলে, বা কলাম লম্বা হলে, ইতিহাস নীরবে ফাঁকা থেকে যায়।

লিপিবদ্ধ কাজ (নমুনা)	কোন অ্যাপ
`attendance.save` · `attendance.correct`	হাজিরা
`assessments.publish` · `.unpublish` · `.marks.save`	মূল্যায়ন
`students.enrol` · `.status` · `.enrolment.end`	ছাত্র
`homework.submit` · `.review` · `.create`	পাঠ
`settings.attendance.update`	নীতি

মোট ২৩টি action নাম · `before` / `after` ৩০০ অক্ষরে কাটা · দেখার অধিকার কেবল `OWNERS`।

যা লিপিবদ্ধ হয় না: লগইন/লগআউট · ব্যর্থ লগইন চেষ্টা · পাসওয়ার্ড বদল · সেটিং ছাড়া অন্যান্য পঠন। অর্থাৎ "কে দেখল" প্রশ্নের উত্তর নেই — কেবল "কে বদলাল"।

শিক্ষকের নোট: দুটি বাক্য তুলনা করুন: "লগ লেখা কখনো কাজ থামাবে না" বনাম "লগ লেখা ব্যর্থ হলে কেউ জানবে না"। দুটোই সত্য — এবং একটির দাম অন্যটি।

প্ল্যাটফর্ম

প্ল্যাটফর্ম কনসোল — একটাই নিয়ম: `is_superuser`

১৮টি ভিউ, ১৮টি ডেকোরেটর, শূন্য ব্যতিক্রম

```
def platform_admin_required(view):
    @wraps(view)
    def _wrapped(request, *args, **kwargs):
        if not request.user.is_superuser:
            raise PermissionDenied(
                "The platform console is for the "
                "system administrator only.")
        return view(request, *args, **kwargs)
    return _wrapped
```

সিদ্ধান্ত	কারণ
workspace-র ভূমিকা নয়	প্ল্যাটফর্ম মালিক কোনো স্কুলের সদস্য নন — সদস্যতা দিয়ে ঢোকা যায় না
কনসোল স্কুলের বাইরে	`/platform/` — No workspace pill, no billing banner
সুপারইউজারও স্কুলের ডেটা পড়েন না	`workspace_view`-তে ব্যতিক্রম নেই — প্ল্যাটফর্ম অ্যাডমিন অন্য স্কুলের ছাত্র-তালিকা খুলতে পারেন না

দুই ক্ষমতা আলাদা: প্ল্যাটফর্ম মালিক স্কুল তৈরি/বন্ধ করতে পারেন, প্ল্যান বদলাতে পারেন — কিন্তু সেই স্কুলের হাজিরা বা নম্বর দেখার ক্ষমতা তাঁর নেই। এটি ইচ্ছাকৃত নকশা, দুর্ঘটনা নয়।

প্রমাণ: `test_only_the_platform_admin_can_open_the_console` — সাধারণ অ্যাডমিন ৪০৩ পান।

শিক্ষকের নোট: "সুপারইউজার সব পারেন" — এই ধারণাটি এখানে ভাঙুন। প্রশ্ন করুন: "সাপোর্ট দিতে গিয়ে ছাত্রের নম্বর দেখা কি ঠিক?" উত্তর: না — প্রয়োজনে সম্মতি নিয়ে, লিপিবদ্ধভাবে, আলাদা প্রক্রিয়ায়।

যাচাই

ছয়টি হাতে-কলমে পরীক্ষা — ১৫ মিনিট

ডেমো চালিয়ে নিজেই যাচাই করুন; প্রতিটির প্রত্যাশিত ফল দেওয়া

#	কী করবেন	যা দেখা উচিত
১	লগআউট করে `/w/springfield/students/` খুলুন	লগইন পাতায় যাবে, `?next=` সহ
২	`teacher@ims.local` দিয়ে ঢুকে অ্যাডমিন-পাতার URL টাইপ করুন	৪০৩ — পর্দা ভাঙে না, বার্তা দেখায়
৩	শিক্ষক দিয়ে অন্য ক্লাসের নম্বর পাতায় যান (pk বদলে)	৪০৩ — "not assigned to this class"
৪	`student@ims.local` দিয়ে অন্য ছাত্রের রেজাল্ট খুলুন	৪০৩ — নিজের রেকর্ডেই সীমা
৫	একটি স্কুলে সাবস্ক্রিপশন মেয়াদোত্তীর্ণ করুন, তারপর সেভ করুন	সেভ হয় না, বার্তা: "your work is not lost"
৬	অচেনা `slug` দিয়ে URL খুলুন	৪০৪ — দুই ক্ষেত্রেই একই বার্তা

নিয়ম: প্রতিটি পরীক্ষা দুবার করুন — একজন অধিকারপ্রাপ্ত ব্যবহারকারী দিয়ে, একজন বঞ্চিত দিয়ে। কেবল প্রথমটি করলে "কাজ করে" প্রমাণ হয়, "বন্ধ" প্রমাণ হয় না।

শিক্ষকের নোট: এই ছয়টি শিক্ষার্থীদের নিজে করতে দিন — ১৫ মিনিটেই বোঝে। ২ ও ৩ নম্বরেই বেশিরভাগের ধারণা বদলায়।

সততা

নিরাপত্তায় যা নেই — ১০টি ফাঁক

প্রতিটির ঝুঁকি এবং প্রতিটির সম্ভাব্য সংশোধন

যা নেই	ঝুঁকি
রেট-লিমিট / লকআউট	পাসওয়ার্ড অনুমান অনন্তকাল চালানো যায়
লগইন অডিট	"কে কখন ঢুকল" বলা যায় না
2FA	পাসওয়ার্ড ফাঁস হলে সব ফাঁস
CSP হেডার	XSS ঘটলে ব্রাউজার-বাধা নেই
ফাইল আকার/ধরন যাচাই	বিপজ্জনক আপলোড সম্ভব

যা নেই	ঝুঁকি
RLS (ডেটাবেজ-স্তরে সীমানা)	ভিউতে ভুল হলে ডেটাবেজ থামায় না
নির্ভরতা-স্ক্যান / Dependabot	প্যাকেজের নিরাপত্তা-প্যাচ অজানা থাকে
এরর-ট্র্যাকিং (Sentry)	ব্যর্থতা চোখে পড়ে ব্যবহারকারীর অভিযোগে
মিডিয়া-ফাইল সুরক্ষা	লিংক জানলে ফাইল দেখা যায়
পাসওয়ার্ড-রিসেটের মেয়াদ কাস্টম	Django ডিফল্ট (৩ দিন) — জেনে রাখা হয়নি

প্রথম তিনটি এক সপ্তাহের কাজ, কিন্তু ডেমো ঠিক চলছে বলে কেউ করে না। **প্রযুক্তিগত ঋণ ভাঙা কোডে নয়, অনুপস্থিত কোডে থাকে** — এই তালিকাই তার প্রমাণ।

শিক্ষকের নোট: শেষ বাক্যটি বোর্ডে লিখুন। শিক্ষার্থীদের জিজ্ঞেস করুন: "এই দশটির মধ্যে কোনটি আজ রাতে ঠিক করা যায়?" — রেট-লিমিট ও লগইন অডিট, দুটোই এক ঘণ্টার কাজ।

8

D4 — লাইফসাইকেল ও অপস

কোড লেখা শেষ, এখন তাকে বাঁচিয়ে রাখা — মাইগ্রেশন, ব্যাকআপ, মনিটরিং, রিলিজ · ২০ স্লাইড